

PDF → JSON PROJECT — MASTER CONTEXT & CURRENT STATE

Updated master reference for continuing the project in a future ChatGPT conversation, maintaining the complete development context, decisions, debugging history, verified checkpoints, current working code, and future-change rules.

Important: This document supersedes earlier master/reference versions where the code differs. Historical code is included only to preserve the development context. The section marked **CURRENT WORKING VERSION** is the source of truth for the present project.

1. PROJECT PURPOSE

The project is a **non-AI PDF → JSON parser** for a known/structured PDF format. It uses normal Python, PyMuPDF (fitz), regular expressions, and JSON.

```
PDF
↓
PyMuPDF
↓
Extract PDF text
↓
Separate Procedure blocks
↓
Regex extraction
↓
Python dictionaries
↓
output.json
```

Required information: Procedure, Time, Dur, STEP IDs, and STEP status grouped into status keys.

Currently supported status values: Success and Failure.

Ignored information: Mode, Source, Priority, STEP description, and the table TIME.

Important limitation: This is not a universal PDF parser. It is designed for PDFs with the established compatible structure. If a future PDF has a materially different layout or labels, the actual extracted text must be inspected before changing the parser.

2. PROJECT ENVIRONMENT

Python: 3.6.5

PyMuPDF / fitz: 1.18.18, 32-bit

IDE: Older PyCharm on the college system

Project folder: samarth (the project can also be placed inside another parent folder such as projects/we_test_bsp; the relative input/output paths remain the same).

No AI / LLM / OCR / ML: The actual extraction is deterministic and uses PyMuPDF + regex + Python data structures + JSON.

3. FINAL PROJECT STRUCTURE

```
samarth/
■■■ main.py
■■■ pdf_parser.py
■■■ input/
■   ■■■ input.pdf
■■■ output.json
```

main.py: overall workflow, PDF reading, procedure separation, dictionary creation, JSON writing, and PDF page-count metadata.

pdf_parser.py: extraction functions for Procedure/Time/Dur and STEP status.

input/input.pdf: the active PDF processed by the current program.

output.json: generated result. It is overwritten on the next run.

No extra Python files such as **utils.py** or **json_generator.py** are required for the current scope. JSON generation is kept in **main.py** because it is only a few lines.

4. ACTUAL PDF STRUCTURE OBSERVED

The tested college PDF contains procedure metadata similar to:

Procedure : abbnm
Source: abbvdksh

Time:31-dec-2025 11:44:13.103
Dur:203.96(s)

Mode:Manual
Priority: Low

The table visually resembles:

STEP ID	STEP	TIME	STATUS
01.0001	<description>	11:44:20	Success
01.0002	<description>	22:22:30	Failure

However, PyMuPDF does not always extract the visual table in exactly this one-line format. Observed extracted layouts included:

```
STEP ID  STEP
TIME
STATUS

01.0002  dfasjjhdf
31:24:56
Success

STEP ID  STEP
TIME
STATUS

01.001   sdfgggggggggg
xashfg   31:24:35      SUCCESS

01.002   sdfgggfdggg
xashfg   31:25:35      SUCCESS

01.0004   dgjkkkkkkkkk
31-06-25 11:44:356   Success

01.0005   dgjkkkkkkkkk
31-07-25 11:44:366   Success
```

This variation was the reason the STEP-status parser had to be made tolerant of line layout while remaining strict about what counts as a STEP ID.

5. DESIRED JSON STRUCTURE

Each procedure is represented by one object:

```
{
  "Procedure": "abbnm",
  "Time": "31-dec-2025 11:44:13.103",
  "Dur": "203.96(s)",
  "status": {
    "success": ["01.0001"],
    "failure": ["01.0002"]
  }
}
```

STEP IDs such as **01.0001** remain strings so leading zeros are preserved.

The full output is a JSON list. After all procedure objects, the current project appends one final PDF information object:

```
{
  "PDFInfo": {
    "No. of Pages read": 6479
  }
}
```

6. DEVELOPMENT HISTORY — WHAT WAS BUILT AND WHY

Initial environment: The project was developed around the older college environment. Python 3.6.5 and PyMuPDF 1.18.18 were established as the working combination.

Initial page test: main.py first opened only page 0 and successfully extracted Procedure, Time, and Dur, but status was empty.

First status investigation: raw page text was printed. This showed that PyMuPDF was splitting STEP data across lines, so the original one-line status regex could not match.

First status fix: the parser stored the STEP ID when it encountered it and later associated a standalone Success/Failure line with that stored STEP ID.

More varied layout: another page showed status on the same line as a time/description, so the parser was changed to look for a status anywhere in the relevant line while retaining the current STEP ID.

Important bug: searching for a number pattern anywhere in a line using `\b(\d+\.\d+)\b` caused pieces of time values such as **44.356** and **44.366** to be interpreted as STEP IDs.

Final status fix: the STEP ID is accepted only when it starts the line, using `^\b(\d+\.\d+)\b`. Once stored, the next matching status is assigned to that STEP ID, and the current STEP ID is cleared.

The corrected status extraction was tested on 3–4 different PDF pages and produced correct STEP IDs and Success/Failure groupings.

7. HISTORICAL STATUS PARSER — OBSOLETE VERSIONS

These are included only so a future ChatGPT session understands why earlier code should **not** be restored.

Original one-line approach — obsolete:

```
match = re.match(r"^\s*(\d+\.\d+)\s+.*?\b(Success|Failure)\s*$", line, re.IGNORECASE)
```

Problem: PyMuPDF often separated STEP ID, description, time, and status across multiple lines.

Overly broad STEP search — obsolete:

```
step_match = re.search(r"\b(\d+\.\d+)\b", line)
```

Problem: this could capture number fragments inside extracted time values. Example: a time containing 44:356 could lead to a false STEP ID such as 44.356.

Current rule: STEP IDs must be recognized at the beginning of a line, then status is associated with the saved STEP ID.

8. FULL-PDF DEVELOPMENT CHECKPOINTS

Step 6 — Read all pages: Completed. The full PDF produced **6,479 pages** and **11,334,398 extracted characters**.

Step 7 — Count procedures: Completed. The full PDF contained **1,254 Procedure: occurrences**.

Step 8 — Separate procedure blocks: The first `re.split` lookahead approach caused a Python 3.6.5 error: **ValueError: split requires a non-empty pattern match**. It was replaced with position-based `re.finditer()` separation. Result: **1,254 procedure blocks**.

Step 9 — Create dictionaries: Completed. **1,254 dictionaries** were created from **1,254 procedure blocks**.

Step 10 — Generate JSON: Completed. `output.json` was generated successfully.

Step 11 — Verification: The user checked several outputs and reported them as correct, including checking different pages and confirming that STEP IDs were being captured rather than time fragments.

Page-processing count feature: Added later. The program now increments `pages_read` inside the page-reading loop and stores the final count in the last **PDFInfo** object.

9. THE `re.split` ERROR AND THE FINAL SOLUTION

The attempted procedure separation used a lookahead with `re.split()`:

```
procedure_blocks = re.split(
    r"(?=Procedure\s*:",
    all_text,
    flags=re.IGNORECASE
)
```

On the established Python 3.6.5 environment this produced:

```
ValueError: split requires a non-empty pattern match
```

The project therefore switched to explicit positions returned by `re.finditer()`:

```
matches = list(
    re.finditer(
        r"Procedure\s*:",
        all_text,
        re.IGNORECASE
    )
)

procedure_blocks = []

for i in range(len(matches)):
    start = matches[i].start()

    if i + 1 < len(matches):
```

```

        end = matches[i + 1].start()
    else:
        end = len(all_text)

    block = all_text[start:end].strip()
    procedure_blocks.append(block)

```

This method successfully separated all 1,254 procedures and is part of the current working main.py.

10. CURRENT WORKING VERSION — pdf_parser.py

This is the current source-of-truth parser. Do not replace it with an older version from the development history.

```

import re

def extract_process_info(text):
    Procedure = ""
    Time = ""
    Dur = ""

    match = re.search(
        r"Procedure\s*:\s*(.*?)\s*(?=Time\s*:$)",
        text,
        re.IGNORECASE | re.MULTILINE
    )

    if match:
        Procedure = match.group(1).strip()

    match = re.search(
        r"Time\s*:\s*(.*?)\s*(?=Mode\s*:$)",
        text,
        re.IGNORECASE | re.MULTILINE
    )

    if match:
        Time = match.group(1).strip()

    match = re.search(
        r"Dur\s*:\s*(.*?)\s*(?=Priority\s*:$)",
        text,
        re.IGNORECASE | re.MULTILINE
    )

    if match:
        Dur = match.group(1).strip()

    return Procedure, Time, Dur

def extract_status(text):
    status_data = {}
    lines = text.splitlines()
    current_step_id = None

    for line in lines:
        line = line.strip()

        step_match = re.match(r"^(\d+\.\d+)\b", line)

        if step_match:
            current_step_id = step_match.group(1)
            continue

        status_match = re.search(
            r"\b(Success|Failure)\b",
            line,
            re.IGNORECASE
        )

        if status_match and current_step_id:
            status = status_match.group(1).lower()

            if status not in status_data:
                status_data[status] = []

            status_data[status].append(current_step_id)
            current_step_id = None

    return status_data

```

11. CURRENT WORKING VERSION — main.py

This is the current source-of-truth main.py, including the actual pages_read counter and PDFInfo output.

```
import fitz
import re
import json

from pdf_parser import extract_process_info, extract_status

pdf = fitz.open("input/input.pdf")

all_text = ""
pages_read = 0

for page in pdf:
    all_text = all_text + page.get_text() + "\n"
    pages_read = pages_read + 1

# Find every Procedure:
matches = list(
    re.finditer(
        r"Procedure\s*:",
        all_text,
        re.IGNORECASE
    )
)

procedure_blocks = []

for i in range(len(matches)):
    start = matches[i].start()

    if i + 1 < len(matches):
        end = matches[i + 1].start()
    else:
        end = len(all_text)

    block = all_text[start:end].strip()
    procedure_blocks.append(block)

# Extract information from every procedure
all_processes = []

for block in procedure_blocks:

    Procedure, Time, Dur = extract_process_info(block)

    status_data = extract_status(block)

    process_data = {
        "Procedure": Procedure,
        "Time": Time,
        "Dur": Dur,
        "status": status_data
    }

    all_processes.append(process_data)

# Add PDF information at the end
pdf_info = {
    "PDFInfo": {
        "No. of Pages read": pages_read
    }
}

all_processes.append(pdf_info)

# Create output.json
with open("output.json", "w") as file:
    json.dump(all_processes, file, indent=4)

print("Total pages:", pages_read)
print("Total text length:", len(all_text))
print("Total procedures:", len(procedure_blocks))
print("Total dictionaries created:", len(all_processes) - 1)
print("No. of Pages read:", pages_read)
print("JSON file created successfully.")

pdf.close()
```

12. WHY pages_read IS USED

The requirement was not merely to display **len(pdf)**. The user specifically wanted to know how many pages the program actually processed.

Therefore the program initializes:

```
pages_read = 0
```

and increments it inside the actual page-reading loop:

```
for page in pdf:
    all_text = all_text + page.get_text() + "\n"
    pages_read = pages_read + 1
```

This means the value in PDFInfo is generated from the loop that actually processes pages, not hard-coded. For the verified college PDF, the resulting value is **6479**.

13. VERIFIED FULL-PDF RESULT

```
Total pages: 6479
Total text length: 11334398
Total procedures: 1254
Total dictionaries created: 1254
No. of Pages read: 6479
JSON file created successfully.
```

These values are the established checkpoints from the tested college PDF.

The JSON therefore contains **1,254 procedure objects**, followed by the final PDFInfo object.

14. WHY THE STATUS PARSER WORKS

The final status algorithm is deliberately simple and based on the observed PDF behavior:

1. Read each extracted line.
2. If a STEP ID appears at the start of the line, save it as `current_step_id`.
3. If the line contains Success or Failure, use the saved `current_step_id`.
4. Add that STEP ID to the corresponding lowercase status list.
5. Clear `current_step_id`.
6. Continue until the procedure block ends.

The strict STEP-ID rule is important because time values can also contain numbers and punctuation. The parser does **not** search for a STEP ID anywhere in the line.

15. FUTURE STATUS VALUES — IMPORTANT CURRENT DECISION

The instructor may provide PDFs where status values are **Success, Progress, Suspend, Failure**, or potentially another status. **Do not change the code speculatively.** First inspect the actual college PDF.

If the real PDF contains only Success and Failure, keep the current parser unchanged.

If Progress and/or Suspend actually appear, update only the status regex in **pdf_parser.py**.

Current code:

```
status_match = re.search(
    r"\b(Success|Failure)\b",
    line,
    re.IGNORECASE
)
```

If the actual PDF requires Progress and Suspend, change it to:

```
status_match = re.search(
    r"\b(Success|Progress|Suspend|Failure)\b",
    line,
    re.IGNORECASE
)
```

The rest of the status logic is already dynamic: it uses the matched status as the dictionary key, converts it to lowercase, creates the list if necessary, and appends the STEP ID.

```
"status": {
    "success": ["01.0001"],
    "progress": ["01.0002"],
    "suspend": ["01.0003"],
    "failure": ["01.0004"]
}
```

For a future new value such as Pending, add it to the same regex. **main.py does not need to change just because a status value is added.**

Because **re.IGNORECASE** is already used, capitalization differences such as Success, SUCCESS, or success do not require another code change.

16. EXACT PROCESS FOR MAKING A CODE CHANGE

When a genuinely new requirement appears:

1. Do not immediately change code.
2. Inspect the actual new PDF or requirement.
3. Identify exactly what differs from the currently supported structure.
4. Keep a backup of the known-good project.
5. Change only the relevant parser/workflow code.
6. Save the file.
7. Run main.py with the actual PDF as input/input.pdf.
8. Check the output against the PDF.
9. If correct, keep the updated version.
10. If incorrect, preserve the error/output and investigate rather than manually editing output.json.

Example: adding a new status changes only the status regex. A new metadata field would require new extraction logic in pdf_parser.py and a corresponding field in process_data in main.py. A different STEP-ID format would require changing the STEP-ID regex only after inspecting the actual PDF.

17. NEW PDF — NORMAL EVERYDAY WORKFLOW

The current project processes **one active PDF at a time** because main.py contains:

```
pdf = fitz.open("input/input.pdf")
```

When a new compatible PDF arrives:

1. Keep the current working project/backup safe.
2. Put the new PDF in the project's input/ folder.
3. Make the PDF you want to process the file name input.pdf.
4. Run main.py.
5. Wait for completion.
6. Open output.json.
7. Check the generated data and PDFInfo.

Other PDFs may be kept in input/ with different filenames, but the current program will not process them unless they are made the active **input.pdf**.

```
input/
■■■ old_college_pdf.pdf
■■■ another_test.pdf
■■■ input.pdf ← active PDF processed by main.py
```

The next run opens output.json in write mode and therefore **overwrites the previous output.json**. If an old result matters, copy/rename it before processing another PDF.

18. BACKUP / GITHUB / SAFE WORKING RULES

The current working version should be treated as the **known-good version**.

Before testing a new PDF or changing parser logic, preserve a copy of the working project. Keep important input PDFs and successful JSON results separately when they may be needed later.

Do not manually edit output.json to make a wrong result look correct. If JSON is wrong, investigate the parser and preserve the evidence.

For GitHub, the master context PDF can be stored as project documentation so a future ChatGPT conversation can use it to reconstruct the project state. The actual source files should remain the authoritative code files.

A practical repository structure can remain simple:

```
project/
■■■ main.py
■■■ pdf_parser.py
■■■ input/
■   ■■■ input.pdf
■■■ output.json
■■■ PDF_to_JSON_Project_MASTER_CONTEXT_UPDATED.pdf
```

If the input PDF is sensitive or not intended for public distribution, do not publish it to a public GitHub repository. The master context document can describe the structure without requiring the real dataset.

19. WHAT NOT TO DO

- Do not add AI, OCR, machine learning, web APIs, databases, or dashboards unless the project requirement explicitly changes.
- Do not add random Python files just to make the project larger.
- Do not restore obsolete status/parser code from earlier development stages.
- Do not use a broad STEP-ID regex that searches anywhere in a line.
- Do not manually edit output.json to hide an extraction error.
- Do not assume a completely different PDF format will work without inspection.
- Do not hard-code the number of pages.
- Do not hard-code the number of procedures.
- Do not add Progress/Suspend merely because they are possible; inspect the real PDF first.

20. LARGE PDF / SCALABILITY UNDERSTANDING

The project has already successfully processed the real **6,479-page** college PDF. That is strong evidence that the approach can handle a large compatible PDF.

For another large PDF with the same general structure, the same workflow is intended to work. There is no project rule limiting the parser to small PDFs.

However, this should not be described as support for arbitrary PDFs of any format. Very large PDFs can require more processing time and memory because the current implementation stores all extracted text in **all_text** before procedure separation.

If a new PDF is structurally different, parser changes may be necessary regardless of its size.

21. ERROR / DEBUGGING RULE

Stop at the first failed checkpoint. Do not continue to later stages if an earlier count or extraction result is wrong.

Example:
Step 7 → 20 procedures
Step 8 → 17 procedure blocks

STOP.
Do not generate JSON.
Investigate procedure separation first.

If Python shows an error, preserve the complete error message. If Python runs but the JSON is wrong, preserve the input PDF and output.json and inspect the actual extracted text/data.

22. HISTORICAL DEVELOPMENT STEPS — COMPACT RECORD

Initial setup: Installed/verified the older Python + PyMuPDF environment and confirmed fitz worked.

First page read: Used `page = pdf[0]` and `page.get_text()` to inspect extracted PDF text.

Metadata extraction: Procedure, Time, and Dur extraction worked before status extraction was fixed.

Status debugging: Printed raw PyMuPDF text to understand the table's actual extracted layout.

Status fix: Stored STEP ID first, then associated later Success/Failure with it.

False STEP-ID bug: Broad number search captured time fragments such as 44.356.

Final STEP-ID rule: Only accept a STEP ID at the beginning of a line.

Multi-page test: Checked several different pages; STEP IDs and statuses were correct.

Full PDF read: Processed 6,479 pages and extracted 11,334,398 characters.

Procedure count: Found 1,254 Procedure markers.

Procedure separation: `re.split` lookahead failed on the environment; `re.finditer` position-based separation replaced it.

Procedure blocks: Successfully separated 1,254 blocks.

Dictionary creation: Created 1,254 procedure dictionaries.

JSON generation: Generated output.json successfully.

Verification: User checked several outputs and reported correct extraction.

Page counter: Added `pages_read` inside the page-reading loop.

PDF metadata: Added final `PDFInfo` object containing No. of Pages read.

23. INSTRUCTOR EXPLANATION

Short explanation: Our project takes a structured PDF as input and extracts the required process information. PyMuPDF is used to open the PDF and extract its text. Python regular expressions identify Procedure, Time, Duration, STEP IDs, and their status. The extracted information is organized into Python dictionaries and written as structured JSON in `output.json`. Fields that are not required, such as Mode, Source, Priority, STEP description, and table TIME, are ignored.

Why PyMuPDF? It is used to open the PDF and extract its text so Python can process the required information.

Why regular expressions? They identify known text patterns such as Procedure, Time, Dur, STEP IDs, and status values.

Why JSON? JSON provides a structured, machine-readable representation of the extracted information.

Why two Python files? `main.py` controls the workflow, while `pdf_parser.py` contains extraction logic. This keeps the code organized and maintainable.

Why no AI? The PDF has a known structure, so deterministic parsing is sufficient and predictable.

Why are STEP IDs strings? They are identifiers, not mathematical values. Strings preserve leading zeros such as 01.0001.

24. COMMON INSTRUCTOR QUESTIONS

Can it handle a 6,479-page PDF?

Yes. The tested college PDF itself contains 6,479 pages and was processed successfully.

Can it handle any PDF?

No. It is designed for compatible structured PDFs. A materially different layout may require parser changes.

Does it use AI?

No. The extraction uses PyMuPDF, Python regex, dictionaries, and JSON.

What happens when a new PDF arrives?

Make it `input/input.pdf`, run `main.py`, and inspect `output.json`.

What happens to the old `output.json`?

The next run overwrites it because the file is opened in write mode.

Why keep `PDFInfo`?

It records the number of pages actually processed by the page-reading loop so it can be compared with the PDF's page count.

Why not store Mode/Source/Priority?

They are not required fields in the current output specification.

What if a new status appears?

Inspect the real PDF and add the status name to the status regex in `pdf_parser.py` if required.

25. CURRENT FINAL CHECKLIST

- Python 3.6.5 environment works
- PyMuPDF 1.18.18 works
- Actual PDF can be opened
- All 6,479 pages were read in the verified college PDF
- 11,334,398 characters were extracted in the verified college PDF
- 1,254 Procedure markers were found
- 1,254 procedure blocks were separated
- 1,254 procedure dictionaries were created
- `output.json` was generated
- Procedure / Time / Dur extraction works
- STEP IDs are captured correctly

- Success / Failure grouping works
- Time fragments are not mistaken for STEP IDs
- STEP IDs remain strings
- PDFInfo records the actual pages_read counter
- New-PDF workflow is understood
- Backup/known-good workflow is understood
- Future status-change process is understood
- Project remains non-AI and intentionally simple

26. CURRENT PROJECT STATE — SOURCE OF TRUTH

Core project: Complete and working for the tested compatible college PDF.

Verified scale: 6,479 pages / 11,334,398 extracted characters / 1,254 procedures.

Current status values: Success and Failure only, unless the real college PDF later proves that additional values are required.

Current JSON ending:

```
{
  "PDFInfo": {
    "No. of Pages read": 6479
  }
}
```

Next practical activity: Test the actual PDFs received from college. If the PDF contains a new status or a different structure, make only the required parser change after inspecting the actual extracted text.

One-sentence project summary: The project extracts selected structured process and STEP-status information from a compatible PDF using PyMuPDF and Python regular expressions, then converts it into structured JSON.

27. FUTURE CHATGPT CONTINUATION INSTRUCTION

If this PDF is uploaded into a future ChatGPT conversation, treat the **CURRENT WORKING VERSION** sections as the source of truth. Preserve the established project decisions, do not revert to obsolete code, and continue from the latest verified state. If the user supplies a new PDF, error, console output, or extracted text, use that actual evidence to decide whether a parser change is required.

When discussing code changes, give the user the exact file, exact section to find, exact replacement code, and exact next action. Keep explanations beginner-friendly and do not introduce unrelated architecture changes.